

Que es Sweet Crush

Es una versión simplificada del juego Candy Crush. Hay un tablero rectangular con fichas de 6 tipos distintos. El jugador elimina fichas individuales; cuando 3 o más fichas iguales quedan alineadas (horizontal o vertical), se eliminan automáticamente. Las fichas de arriba caen para llenar los huecos y se generan fichas nuevas. Si esas nuevas fichas forman combinaciones, también se eliminan (cascada).

Que lo hace diferente a un juego normal

La restricción clave es la representación interna: **cada ficha ocupa 3 bits**, sin padding, sin alineación a byte ni estructuras auxiliares permanentes. El tablero se comporta como una tira continua de bits empaquetada en bytes.

Esto significa que, para cualquier operación sobre el tablero -leer, escribir, copiar o modificar una ficha-, se necesita realizar aritmética de bits manualmente.

Por qué 3 bits

Con 3 bits puedes representar 8 valores distintos ($2^3 = 8$). Necesito 6 para las fichas A-F, y me sobran 2 para estados como VACÍO y ESPECIAL. Es el mínimo de bits que cubre las necesidades.

Si usara 2 bits, solo tendría 4 valores (no alcanza para 6 fichas). Si usara 4 bits, desperdiciaría la memoria innecesariamente.

1. Estructura y organización del proyecto

El desarrollo de Sweet Crush requiere una organización estructurada del proyecto, separando las responsabilidades que intervienen en el funcionamiento del juego. La propuesta contempla dividir el código en módulos dedicados a aspectos específicos, como el manejo de bits, la creación y administración del tablero, la lógica del juego, las modificaciones de filas y columnas y la interacción con Qt. Esta división permite comprender el impacto de cada parte en el funcionamiento general y facilita conectar correctamente estos elementos sin mezclar sus responsabilidades, especialmente al trabajar con una representación del tablero distinta de la convencional. Entender esta estructura ha sido importante para desarrollar el proyecto de forma progresiva y facilitar la identificación y corrección de errores.

2. Dificultades con la lógica y los conceptos de programación

Uno de los mayores retos del proyecto ha sido construir y comprender la lógica necesaria para que el juego funcione correctamente. No se trata solo de mostrar un tablero, sino de establecer las bases para detectar combinaciones, eliminar fichas, aplicar la gravedad, generar nuevas fichas y controlar las cascadas. También ha sido difícil trabajar con operadores a nivel de bits, ya que es necesario comprender cómo funcionan los desplazamientos, las máscaras y las operaciones lógicas para leer y modificar una ficha almacenada en solo 3 bits. Además, ha sido necesario reforzar el manejo de punteros y de memoria dinámica, comprendiendo cómo acceder a la información del búfer y cómo reservar y liberar correctamente la memoria. Estos conceptos representan una parte importante del aprendizaje durante la construcción del proyecto.

3. Desarrollo progresivo y uso de Git

Otro aspecto importante del proyecto ha sido aprender a desarrollar de forma progresiva con Git como herramienta de control de versiones. La intención no es realizar todo el proyecto de una vez, sino mostrar la evolución en la construcción de la solución y cómo cada cambio afecta al avance: empezar por el análisis del problema y la representación de los bits, seguir con la lógica del tablero e integrar después las funciones más complejas y la interfaz en Qt. Así se registran los cambios, se identifica cuándo se introducen errores y se recuperan versiones anteriores cuando sea necesario. Además, los commits con cambios específicos ayudan a organizar el trabajo y a comprender mejor cómo cada parte del proyecto se construye sobre la anterior.